

Report

Structure

- bank_server.c
- bank_client.c
- helpers.c
- helpers.h
- makefile

Build with *make*, run with *bank_server.c <tcp|udp> <port>* and then *bank_client.c <tcp|udp> <port>*. Right away, you can start sending commands through the client.

Sections 1 & 3: TCP Client & UDP Client

I decided to not split my client into 2 distinct paths. Instead, I used if statements to split the differences in TCP/UDP logic at key parts.

Part 1: main()

```
int main(int argc, char** argv) {
    int status = NO_ERROR;

    const char help_msg[] = "Usage: ./bank_client <tcp|udp> <port>\n";

    Args args = parse_args(argc, argv, &status);
    if (status != NO_ERROR) {
        printf(help_msg);
        goto cleanup;
    }

    int sock_fd = socket(AF_INET, args.sock_type, 0);
    if (sock_fd < 0) {
        perror("socket()\n");
        goto cleanup;
    }

    connection_loop(sock_fd, args.port, args.sock_type == SOCK_STREAM);

cleanup:
    close_cleanly(&sock_fd);

    return status;
}
```

Part 2: Setting up connection loop (important for TCP, not for UDP)

```
15 int connection_loop(int sock_fd, port_t server_port, int is_tcp) {
16     int status = NO_ERROR;
17
18     struct sockaddr_in server_addr = get_server_sock(server_port, 0);
19     if (is_tcp) {
20         status = connect(sock_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
21         if (status < 0) {
22             perror("connect()\n");
23             goto cleanup;
24         }
25     }
26
27     char user_input[MSG_MAX_LEN];
28     char server_output[MSG_MAX_LEN];
```

Part 3: Connection loop (both TCP and UDP)

```
29     while (1) {
30         fgets(user_input, MSG_MAX_LEN, stdin);
31         int msg_len = strlen(user_input);
32
33         if (msg_len == 0) {
34             break;
35         }
36
37         long bytes_sent = 0;
38         while (bytes_sent < msg_len) {
39             struct sockaddr *sockaddr_server_addr = (is_tcp ? NULL : (struct sockaddr *)&server_addr);
40             socklen_t len = (is_tcp ? 0 : sizeof(server_addr));
41             status = sendto(sock_fd, user_input + bytes_sent, msg_len, 0, sockaddr_server_addr, len);
42             if (status < 0 && errno != EINTR) {
43                 perror("send()\n");
44                 goto cleanup;
45             } else {
46                 bytes_sent += status;
47             }
48         }
49     }
```

```

50     long bytes_received = 0;
51     size_t recv_size = is_tcp ? 1 : MSG_MAX_LEN;
52     while (1) {
53         status = recv(sock_fd, server_output + bytes_received, recv_size, 0);
54         if (status < 0 && errno != EINTR) {
55             perror("recv()\n");
56             goto cleanup;
57         } else if (status == 0 && is_tcp) {
58             status = ERROR;
59             printf(SERVER_CLOSED_ERR);
60             goto cleanup;
61         } else if (status >= MSG_MAX_LEN) {
62             status = ERROR;
63             printf(MSG_OVERFLOW_ERR);
64             goto cleanup;
65         } else if (status > 0) {
66             if (server_output[bytes_received + status - 1] == '\n') {
67                 bytes_received += status;
68                 break;
69             }
70             bytes_received += status;
71         }
72     }
73     server_output[bytes_received] = '\0';

```

```

74
75     printf("%s", server_output);
76
77     int reply_is_bye = strstr(server_output, BANK_CODE_BYE) == server_output;
78     if (reply_is_bye) {
79         printf(BYE_MSG);
80         goto cleanup;
81     }
82 }
83
84 cleanup:
85     return status;
86 }

```

Section 2: TCP Server

After making the client, I realized it would be a lot more organized to just split the TCP/UDP code into two functions, so this is just for TCP.

Part 1: TCP Set up

```
53 int accept_tcp(int sock_fd) {
54     int status = NO_ERROR;
55     int balance = STARTING_BALANCE;
56
57     char msg_buffer[MSG_MAX_LEN];
58     char server_response[MSG_MAX_LEN];
59
60     status = listen(sock_fd, 1);
61     if (status < 0) {
62         perror("listen()\n");
63         goto cleanup;
64     }
65
66     int client_fd = 0;
67     while (client_fd == 0 || errno == EINTR) {
68         client_fd = accept(sock_fd, NULL, NULL);
69     }
70     if (client_fd < 0) {
71         perror("accept()\n");
72         goto cleanup;
73     }
```

Part 2: Processing Input

```
75     while (1) {
76         long bytes_received = 0;
77         while (1) {
78             status = recv(client_fd, msg_buffer + bytes_received, 1, 0);
79             if (status < 0 && errno != EINTR) {
80                 perror("recv()\n");
81                 goto cleanup;
82             } else if (status == 0) {
83                 status = ERROR;
84                 goto cleanup;
85             } else if (status > 0) {
86                 if (msg_buffer[bytes_received++] == '\n') {
87                     break;
88                 }
89             }
90         }
91         msg_buffer[bytes_received - 1] = '\0';
92
93         status = process_request(&balance, msg_buffer, server_response);
94         if (status < 0) {
95             perror("process_request()\n");
96         }
```

Part 3: Send Response

```

98     long bytes_sent = 0;
99     int msg_len = strlen(server_response);
100     while (bytes_sent < msg_len) {
101         status = send(client_fd, server_response + bytes_sent, msg_len, 0);
102         if (status < 0 && errno != EINTR) {
103             perror("send()\n");
104             goto cleanup;
105         } else {
106             bytes_sent += status;
107         }
108     }
109
110     if (balance == -1) {
111         // QUIT command
112         break;
113     }
114 }
115
116 cleanup:
117     close_cleanly(&client_fd);
118
119     return status;

```

Section 4: UDP Server

Part 1: Processing Input

```

122 int accept_udp(int sock_fd) {
123     int status = NO_ERROR;
124     int balance = STARTING_BALANCE;
125
126     char msg_buffer[MSG_MAX_LEN];
127     char server_response[MSG_MAX_LEN];
128
129     struct sockaddr_in client_addr;
130     socklen_t client_addr_len = sizeof(client_addr);
131
132     while (1) {
133         long bytes_received = 0;
134         while (1) {
135             status = recvfrom(sock_fd, msg_buffer, MSG_MAX_LEN, 0, (struct sockaddr *)&client_addr, &client_addr_len);
136             if (status < 0 && errno != EINTR) {
137                 perror("recv()\n");
138                 goto cleanup;
139             } else if (status >= 0) {
140                 bytes_received = status;
141                 break;
142             }
143         }
144         msg_buffer[bytes_received - 1] = '\0';
145
146         status = process_request(&balance, msg_buffer, server_response);
147         if (status < 0) {
148             perror("process_request()\n");
149         }

```

Part 2: Send Response

```
151     long bytes_sent = 0;
152     int msg_len = strlen(server_response);
153     while (bytes_sent < msg_len) {
154         status = sendto(sock_fd, server_response, msg_len, 0, (struct sockaddr *)&client_addr, sizeof(client_addr));
155         if (status < 0 && errno != EINTR) {
156             perror("send()\n");
157             goto cleanup;
158         } else {
159             bytes_sent += status;
160         }
161     }
162
163     if (balance == -1) {
164         // QUIT command
165         break;
166     }
167 }
168
169 cleanup:
170     return status;
171 }
```

Section 5: Tests

TCP Client/Server Tests:

```
● romqrocha@DESKTOP-3POQ0V1:~/7005/lab3$ ./bank_server tcp
2121
❖ romqrocha@DESKTOP-3POQ0V1:~/7005/lab3$

● romqrocha@DESKTOP-3POQ0V1:~/7005/lab3$ ./bank_client tcp
2121
DEPOSIT 100
OK BALANCE 1100
WITHDRAW 100
OK BALANCE 1000
BALANCE
OK BALANCE 1000
WITHDRAW 10000
ERR INSUFFICIENT_FUNDS
BALANCE
OK BALANCE 1000
QWERTYUIOP
ERR INVALID_CMD
QUIT
BYE
'BYE' received. Cleanly ending the connection.
❖ romqrocha@DESKTOP-3POQ0V1:~/7005/lab3$
```

UDP Client/Server Tests:

```
● romqrocha@DESKTOP-3POQ0V1:~/7005/lab3$ ./bank_server udp
2121
❖ romqrocha@DESKTOP-3POQ0V1:~/7005/lab3$

● romqrocha@DESKTOP-3POQ0V1:~/7005/lab3$ ./bank_client udp
2121
DEPOSIT 100
OK BALANCE 1100
WITHDRAW 100
OK BALANCE 1000
BALANCE
OK BALANCE 1000
WITHDRAW 10000
ERR INSUFFICIENT_FUNDS
BALANCE
OK BALANCE 1000
QUIT
BYE
'BYE' received. Cleanly ending the connection.
❖ romqrocha@DESKTOP-3POQ0V1:~/7005/lab3$
```